FPGA-Accelerated Neural Network for Wildfire Pixel Classification

Thesis

by

Your Name

Department of Electrical and Computer Engineering
University Name
City, Country
Month Year

Supervisor: Dr. Supervisor Name
Co-Supervisor: Dr. Co-Supervisor Name

Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Electrical and Computer Engineering.

Abstract

This thesis presents the design, training, and hardware implementation of a compact multilayer perceptron (MLP) for per-pixel wildfire detection on RGB imagery. A Python pipeline trains a fixed-point friendly network using remotely-sensed wildfire and non-fire image patches. The trained weights, biases, and activation lookup tables are exported for FPGA synthesis. The hardware architecture is described in VHDL and integrated in Xilinx Vivado targeting an UltraScale device. Results demonstrate real-time throughput with competitive accuracy under fixed-point quantization.

Key contributions include: (i) dataset preparation and a reproducible training pipeline, (ii) fixed-point export flows and an activation LUT generation methodology, (iii) a streaming VHDL architecture for inference with resource/performance measurements, and (iv) experimental evaluation on test imagery and comparison to floating-point baselines.

Keywords: wildfire detection, FPGA, VHDL, fixed-point neural networks, MLP, LUT-based activation

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Supervisor Name, for guidance and support throughout this work. I also thank my colleagues and family for their encouragement. Finally, I acknowledge the use of open-source tools and datasets that enabled this research.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Wildfires pose severe environmental and socioeconomic risks. Early and reliable detection enables timely response and mitigation. Modern computer vision systems leverage machine learning to classify imagery; however, deploying such models for real-time operation at the edge often requires careful optimization. Field-programmable gate arrays (FPGAs) offer an attractive platform due to their parallelism, low latency, and energy efficiency.

This thesis investigates an FPGA-based implementation of a compact multilayer perceptron (MLP) that performs per-pixel wildfire classification from RGB inputs. The contributions of this work are:

- A reproducible training pipeline for a lightweight MLP suitable for fixed-point inference.
- A quantization-aware export flow that produces weights, biases, and activation lookup tables compatible with VHDL.
- A streaming VHDL architecture integrating ‘multiplier‘, ‘neuron‘, ‘nn_rgb‘, and ‘sigmoid_lut‘ modules for real-time inference.
- An evaluation of accuracy, throughput, and FPGA resource utilization on a Xilinx UltraScale device.

1.1 Thesis Structure

Chapter **??** reviews background on wildfire detection, neural networks, fixed-point arithmetic, and FPGA architectures. Chapter **??** describes the dataset, preprocessing, network design, and the training/export toolchain. Chapter **??** details the VHDL design and integration in Vivado. Chapter **??** presents the experimental evaluation. Chapter **??** concludes and outlines future work.

Chapter 2

Background

This chapter summarizes key concepts relevant to this thesis.

2.1 Wildfire Detection in Imagery

Approaches range from handcrafted features (e.g., color-based rules) to data-driven classifiers and deep neural networks. Edge deployment motivates compact models and efficient inference.

2.2 Multilayer Perceptrons

An MLP comprises fully connected layers with nonlinear activations. For per-pixel classification, a small network can map RGB inputs to a binary fire/non-fire probability. Quantization and activation approximations (e.g., lookup tables) enable efficient hardware mapping.

2.3 Fixed-Point Arithmetic and LUT Activations

Fixed-point formats reduce memory usage and DSP requirements at modest accuracy cost. Sigmoid or similar activations can be approximated using lookup tables (LUTs) generated offline and stored in block RAM.

2.4 FPGAs and VHDL

FPGAs provide reconfigurable logic, DSP slices, and block RAMs. VHDL is used to describe the hardware. Vendor tools like Xilinx Vivado perform synthesis, placement, routing, and bitstream generation.

2.5 Related Work

Prior work has demonstrated neural inference on FPGAs using fixed-point quantization, pipelining, and memory tiling. This thesis adopts similar principles for a simple MLP.

Chapter 3

Dataset and Methods

This chapter documents the dataset, preprocessing, network architecture, training, and export flows.

3.1 Dataset

The repository contains a ‘data/wildfire‘ folder with ‘Image‘ and ‘Segmentation_Mask‘ subfolders, each split into ‘Fire‘ and ‘Not_Fire‘. Per-pixel labels are derived from the masks. Example paths: ‘data/wildfire/Image/Fire/‘, ‘data/wildfire/Segmentation$_Mask/Not_Fire/$‘.

3.2 Preprocessing

RGB pixels are normalized and optionally augmented. Patches or streaming pixels can be used to build training samples balancing classes.

3.3 Network Architecture

A compact MLP maps three inputs (R, G, B) to a scalar fire probability. Hidden layer width is selected to meet accuracy and hardware constraints. The activation function is chosen to be sigmoid for hardware-friendly LUT approximation.

3.4 Training Pipeline

The ‘python/train$_mlp.py$‘$scripttrainsthemodelandsavesartifactsunder$‘$python/artifacts/$‘($e.g.$, ‘$W1.$

3.5 Fixed-Point Export

Scripts ‘python/export$_fixedpoint.py$‘, ‘$python/gen_sigmoid_lut.py$‘, and‘$python/export_to_xilinx.py$‘$quanti$

3.6 Test Vectors

The ‘python/make$_t$$e$$s$$t$$_v$$e$$c$$t$$o$$r$$s$.$p$$y$‘$u$$t$$i$$l$$i$$t$$y$$e$$x$$p$$o$$r$$t$$s$$g$$o$$l$$d$$e$$n$$v$$e$$c$$t$$o$$r$$s$$t$$o$‘$p$$y$$t$$h$$o$$n$/$a$$r$$t$$i$$f$$a$$c$$t$$s$/$t$$e$$s$$t$$v$$e$$c$$t$$o$$r$$s$/‘($e$.$g$.,‘$p$

Chapter 4

FPGA Implementation

This chapter details the VHDL design and Vivado integration.

4.1 Architecture Overview

The design comprises parameterized ‘multiplier‘ units, ‘neuron‘ blocks aggregating weighted sums with bias, and a top-level ‘nn_rgb‘ module orchestrating layers. The ‘sigmoid_lut‘/‘sigmoid_IP‘ provides activation using a precomputed lookup table stored in block RAM.

4.2 VHDL Modules

Key files in ‘VHDL/‘ include:

- ‘config.vhd‘: Global configuration (bit widths, scaling factors).
- ‘multiplier.vhd‘: Fixed-point multiply stage using DSP resources.
- ‘neuron.vhd‘: Accumulation and bias addition; interfaces to activation.
- ‘nn_rgb.vhd‘: Top entity connecting layers and streaming interfaces.
- ‘sigmoid_lut.vhd‘ and ‘sigmoid_IP.vhd‘: Activation lookup.
- ‘tb_nn_rgb.vhd‘, ‘sim_nn_rgb.vhd‘: Testbench and simulation harness.

4.3 Simulation

ModelSim/Questa simulation artifacts are under ‘sim/‘. Provided test vectors allow functional verification against golden outputs. Waveforms and logs (e.g., ‘vsim.wlf‘) support debugging.

4.4 Synthesis and Implementation

The Vivado project ‘vivado$_t$$hesis/thesis_u$$ltrascale_1$/‘$targetsanUltraScaledevice.Post-implementationreports(timing, utilization, clocking)arestoredunder‘thesis_ultrascale_1.runs/‘.Thedes timethroughputforstreamingpixels.$

4.5 Resource/Performance Considerations

Design parameters (bit widths, layer sizes) affect LUTs, FFs, BRAM, and DSP usage. Pipelining and parallelization trade latency for throughput. Activation via LUT reduces DSP demand versus CORDIC or piecewise-linear implementations.

Chapter 5

Results and Evaluation

This chapter presents classification performance, fixed-/float-point comparisons, and FPGA resource/utilization metrics.

5.1 Classification Metrics

Report accuracy, precision, recall, F1, ROC AUC on a held-out test set. Visualize examples with true positives/negatives and errors.

5.2 Fixed- vs Floating-Point

Quantify accuracy deltas due to quantization. Compare CPU float inference to FPGA fixed-point outputs using golden vectors.

5.3 FPGA Utilization and Timing

Summarize post-implementation utilization (LUT, FF, BRAM, DSP) and timing (WNS, Fmax). Include throughput (pixels/s) and latency estimates.

5.4 Ablations

Optional: Evaluate layer width, bit width, and activation LUT resolution.

Chapter 6

Conclusion and Future Work

This thesis demonstrated a compact, fixed-point MLP for per-pixel wildfire detection implemented on an FPGA. The system achieves real-time throughput with competitive accuracy while using modest resources. The toolchain from training to hardware export enables reproducible development and verification.

Future extensions include: (i) multi-spectral inputs and temporal cues, (ii) convolutional feature extractors, (iii) on-board learning or adaptive thresholds, and (iv) deployment on embedded SoC platforms with camera interfaces.

Appendix A

Repository Structure and Artifacts

This appendix summarizes relevant folders and artifacts used in the thesis.

- ‘python/‘: Training and export scripts (‘train$_m lp.py$‘, ‘$export_f ixedpoint.py$‘, ‘$gen_s igmoid_l ut.py$‘, et

- ‘python/artifacts/‘: Trained parameters and plots (‘W1.npy‘, ‘W2.npy‘, ‘b1.npy‘, ‘b2.npy‘, ‘plots/‘).
- ‘coeffs/‘: Activation LUT in text/COE formats used by the FPGA design.
- ‘VHDL/‘: Hardware description modules, testbenches, and top-level entities.
- ‘sim/‘: Simulation workspace and waveforms.
- ‘vivado$_t hesis/$‘ : $Vivadoprojectandimplementation/synthesisruns$.